



Programmable Property-Based Testing

ALPEREN KELES, University of Maryland, USA

JUSTINE FRANK, University of Maryland, USA

CEREN MERT, University of Maryland, USA

HARRISON GOLDSTEIN, SUNY University at Buffalo, USA

LEONIDAS LAMPROPOULOS, University of Maryland, USA

Property-based testing (PBT) is a popular technique for establishing confidence in software, where users write *properties*—i.e. executable specifications—that can be checked many times in a loop by a testing framework. In modern PBT frameworks, properties are usually written in *shallowly embedded* domain-specific languages, and their definition is tightly coupled to the way they are tested. Such frameworks often provide convenient configuration options to customize aspects of the testing process, but users are limited to precisely what library authors had the prescience to allow for when developing the framework; if they want more flexibility, they may need to write a new framework from scratch.

We propose a new, deeper language for properties based on a mixed embedding that we call *deferred binding abstract syntax*, which reifies properties as a data structure and decouples them from the property runners that execute them. We implement this language in Rocq and Racket, leveraging the power of dependent and dynamic types, respectively. Finally, we showcase the flexibility of this new approach by implementing a variety of property runners in a shared framework, highlighting domain-specific testing improvements that can be unlocked by more programmable testing.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**.

Additional Key Words and Phrases: property-based testing, QuickChick, property language, DSL, deep embedding

ACM Reference Format:

Alperen Keles, Justine Frank, Ceren Mert, Harrison Goldstein, and Leonidas Lampropoulos. 2026. Programmable Property-Based Testing. *Proc. ACM Program. Lang.* 10, ICFP, Article 287 (August 2026), 32 pages. <https://doi.org/10.1145/3828685>

1 Introduction

In property-based testing [Claessen and Hughes 2000], users build confidence in their code using *properties* that describe what it means for a program to be correct, expressed in the form of universally quantified executable predicates: e.g. for all expressions, evaluating them with and without optimizations yields the same result. Property-based testing frameworks provide two main pieces that facilitate testing: a *property language* and a *property runner*. The property language is the API that allows users to express these executable predicates, and it provides limited ways to configure testing behavior (such as how to generate or pretty print test inputs); the property runner is the way a framework actually runs a property (taking any configuration into account) to evaluate the system under test.

Authors' Contact Information: Alperen Keles, University of Maryland, College Park, USA, akeles@umd.edu; Justine Frank, University of Maryland, College Park, USA, jpf Frank@umd.edu; Ceren Mert, University of Maryland, College Park, USA, cmert@umd.edu; Harrison Goldstein, SUNY University at Buffalo, Buffalo, USA, hgoldste@buffalo.edu; Leonidas Lampropoulos, University of Maryland, College Park, USA, leonidas@umd.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2475-1421/2026/8-ART287

<https://doi.org/10.1145/3828685>

Consider, for example, the optimization correctness property above in Haskell’s QuickCheck:

```
prop_eval :: Property
prop_eval =
  forAllShrinkShow gen shrink show
  (\e → eval e == eval (optimize e))

gen :: Gen Exp
shrink :: Exp -> [Exp]
show :: Exp -> String
eval, optimize :: Exp -> Exp
```

Given some type of expressions `Exp`, users provide (or automatically derive [Lampropoulos et al. 2018]): (1) a generator for expressions `gen`, that is, a function from some random seed to a concrete `Exp`; (2) a shrinking function `shrink`, a function from an expression to a list of potentially smaller expressions for minimization purposes; (3) a printing function `show`, in order to report counterexamples to the user; (4) and a predicate on expressions, which in this case is an **anonymous function** that given an expression `e`, evaluates it with and without optimizations, and checks that the results are equal. To create a `Property` that QuickCheck can test, users can leverage the `forAllShrinkShow` combinator from QuickCheck’s property language API to put everything together, or let typeclasses take care of this final assembly. QuickCheck, and most PBT frameworks, arm users with a variety of ways to configure many aspects of testing: from implementing their own hand-tuned generator that produces a better distribution of inputs to collecting bespoke statistics to gauge the distribution of inputs produced.

Unfortunately, there is one aspect of testing that is baked into these frameworks and can fundamentally not be configured: the property runner—the very way properties are tested. The core of QuickCheck’s property runner is surprisingly simple, as we depict pictorially in Fig. 1. Expressions are continuously generated and checked until a counterexample is found; then the counterexample is repeatedly shrunk until the smallest expression that invalidates the property is found. The

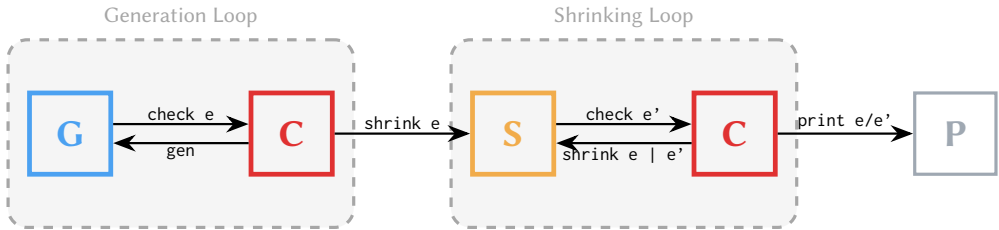


Fig. 1. An Abstract Representation of QuickCheck’s Property Runner

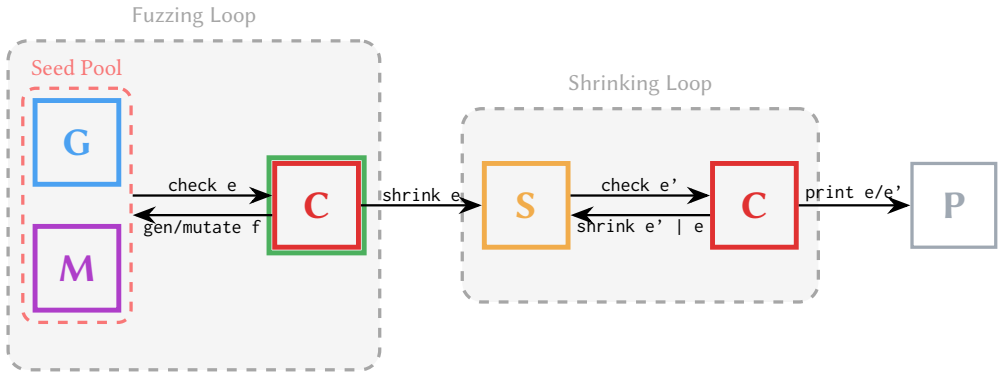


Fig. 2. An Abstract Representation of FuzzChick’s Property Runner

(locally) minimal counterexample is then presented to the user. While some details of these two loops can be configured (e.g., the number of tests to run, how long to spend on shrinking, etc.), users cannot configure the core functionality of the loop itself. As we will see, the structure of the testing loop is baked into the way properties are represented internally as a shallow embedding.

But there are good reasons that a user might want to change the testing loop! Recent advances offer compelling alternative structures. For example, the literature is rife with testing approaches inspired by coverage-guided fuzzing, where rather than generating new inputs from scratch at each iteration of the loop, the runner instead keeps track of inputs that led execution down a novel path, and mutates those in the hopes of uncovering yet more interesting paths [Dolan 2017; Hatfield-Dodds and contributors 2025; Lampropoulos et al. 2019]. If we were to draw a diagram for such a system, such as FuzzChick [Lampropoulos et al. 2019], we might get something like the one that appears in Fig. 2.

Despite the similarity of these two diagrams, the implementations of these approaches are almost entirely distinct: no reuse of components for the implementors of the frameworks, no reuse of properties for the users. As we will discuss in detail, the prevalent QuickCheck-based design of the property language bakes the generation and minimization loops into an opaque representation of properties using a shallow embedding that only allows for a single predefined interpretation: executing the loop of Fig. 1. In this work, we challenge this design decision, and introduce a new way of representing properties that allows property runners to be written at the “user-level”, without needing to dive into or modify library internals.

To that end, we turn to a deeper, but not entirely deep, embedding for the representation of properties, designing a language that can be reified as a data structure and then interpreted in different ways [Chlipala 2008; Goldstein and Pierce 2022; Hudak 1996; Prinz et al. 2022]. We introduce a novel style of mixed embeddings that is particularly well-suited for representing properties for testing, which we call *deferred binding abstract syntax*: rather than binding a variable once at the site of its universal quantification, we will instead bind it at every one of its use sites. This representation allows us to fully *decouple the specification from the runner*: the property itself expresses only the specification and the runner can be programmed by the user to interpret the property—allowing for maximum programmability.

We offer the following contributions:

- We introduce a new style of mixed embeddings, which we call *deferred binding abstract syntax*, and use it to define a property language that allows for arbitrary re-interpretation of the way properties are executed (§ 3).
- We implement our property language API in the QuickChick property-based testing framework for Rocq, leveraging dependent types to ensure that properties are well-formed while retaining a surface syntax close to existing property APIs (§ 4).
- We implement our property language API as part of a new PBT framework in Racket, leveraging dynamic typing and macros to hide the internal data structure, providing an identical user interface to existing libraries, while enabling flexibility through the deeper embedding (§ 5).
- We thoroughly evaluate our approach in three ways: First, we demonstrate the flexibility of this new language by implementing a variety of complex runners from the recent literature—including ones with coverage-guided fuzzing and context-sensitive shrinking—all in user code. Then, we compare the performance of our implementation to existing frameworks, showing negligible overhead. Finally, we showcase how the flexibility of our approach allows experiments to be expressed as changes to reusable runners and runner components, by carrying out three experiments to fine tune testing aspects.

We conclude with related (§ 7) and future (§ 8) work.

2 Background: Property Runners

We begin by motivating the need for a programmable property-based testing framework by exploring a wide variety of property runners from the literature, highlighting their similarities and differences. We depict the runners using abstract representations (like the ones in the introduction) of the dataflow between different *components* of the runners, such as **generators**, **shrinkers**, or **printers**. Crucially, existing implementations of the runners we discuss are distinct, spread across different libraries in different languages. This section presents our attempt at putting them within a single conceptual framework; later we will show how to implement all of them in a single framework.

2.1 Simple Generational Property Runner

We have already discussed the quintessential property runner of Fig. 3, as proposed by QuickCheck [Claessen and Hughes 2000]. Looking a bit more closely, the runner consists of two stages: the first is a generate-and-check stage that repeatedly generates random inputs and tests them against the stated property until a maximum limit on the number of tests is reached, or until an input falsifying the property—i.e, a bug—is found. If such a bug is found, the second stage of the runner is a shrink-and-check stage that repeatedly tries smaller inputs (produced by a user-provided “shrinking” function) until a local minimum—an input that can no longer be shrunk—is reached.

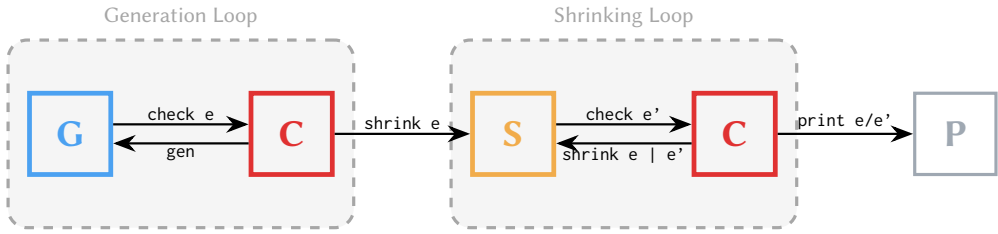


Fig. 3. The QuickCheck-style property runner from Fig. 1, repeated here for comparison with alternative runner structures.

In principle, such a runner could be implemented using two straightforward modular loops that could be composed together in sequence. However, QuickCheck’s design has the shrinking behavior built into the way properties are executed: when a test input is generated, a tree of potentially smaller counterexamples is lazily generated along the way, and is used for minimization. As such, changing the shrinking behavior of QuickCheck, for example to introduce integrated shrinking as we will discuss right below, necessitates deep changes to the property representation and the property runner, and as a consequence often a new framework.

2.2 Integrated Shrinking Property Runner

Integrated shrinking emerges as a solution to a prevailing problem of type-based shrinking: generators are usually tuned to only produce inputs that satisfy implicit or explicit validity constraints, and type-based shrinking makes it surprisingly easy to not take such constraints into account, leading to minimized but invalid counterexamples. A well-known instance of this issue manifested in the line of work on Csmith and C-Reduce [Regehr et al. 2012; Yang et al. 2011], where C programs were generated as to not exhibit undefined behavior, but shrinking based on the C program grammar alone was very likely to introduce undefined behavior during minimization, producing uninteresting counterexamples unless the shrinking process is guarded by external tools for detecting undefined behavior.

Integrated shrinking solves this problem by removing the shrinker from the equation altogether, instead reusing the generators themselves to carry out the minimization. Generation can be thought of as a process where random bytes are parsed into structured test cases [Goldstein and Pierce 2022]; in integrated shrinking, rather than minimizing the structured output of generators, frameworks minimize the randomness that goes into them. As a result, any validity constraints that are built into a generator are preserved by construction during shrinking. This method of integrated shrinking, depicted in Fig. 4, can be found in many frameworks such as Python’s Hypothesis [Maciver and Donaldson 2020] and Haskell’s Hedgehog [Stanley 2019] and Falsify [de Vries 2023].

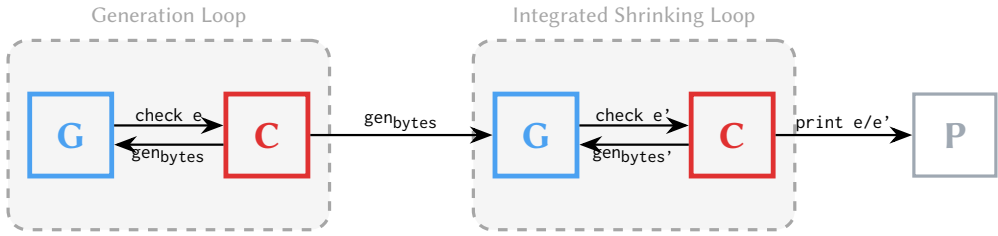


Fig. 4. An Abstract Representation of the Integrated Shrinking Property Runner

Traditional and integrated shrinking offer an interesting trade-off. Integrated shrinking removes the burden of writing shrinkers (and especially ones that preserve any input invariants) altogether, while QuickCheck’s traditional approach can often lead to substantially smaller inputs. In other words, different situations call for different approaches. Unfortunately, “baking in” the way inputs are minimized into the way properties are represented and executed means that to take advantage of a different approach users might have to switch frameworks, or even languages entirely! In § 6.4 we will show how the flexibility of our proposed representation allows users to leverage whichever approach is more appropriate, exploring this trade-off.

2.3 Coverage-Guided (Fuzzing) Property Runner

As we already discussed in the introduction, coverage-guided fuzzing, popularized by AFL [Fioraldi et al. 2020], is an effective random testing technique which makes major, breaking changes to the generation loop, as shown in Fig. 5. In more detail, the predicate that is being checked is instrumented, allows the runner to keep track of the branches that were covered during execution; then, the runner uses a genetic-style algorithm that attempts to maximize this coverage. In most instantiations, the runner keeps track of a *seed pool*: a corpus of inputs that led execution down interesting, previously unseen paths. Inputs are then selected from this pool, mutated randomly, until a bug (or, more commonly, simply a crash) is found.

The effectiveness of fuzzing has led to multiple attempts to incorporate it into property-based testing: from early ones like OCaml’s Crowbar [Dolan 2017], Rocq’s FuzzChick [Lampropoulos et al. 2019], and Java’s JQF [Padhye et al. 2019a], to more recent ones like HypoFuzz [Hatfield-Dodds and contributors 2025]. Crucially, each such attempt was, by necessity, either a completely new framework (Crowbar, JQF), or a major redesign of an existing one (QuickChick for FuzzChick, Hypothesis for HypoFuzz) that required significant changes to internals in order to even offer an alternative to its existing testing strategy.

Worse, the difficulty of incorporating such changes into existing frameworks also makes it difficult to experiment with new ideas. For example, the search strategy of a fuzzing loop (how seeds are selected for mutation) has seen significant research interest [Böhme et al. 2019; Böhme et al. 2017; Lemieux and Sen 2018]. However, when adapting property-based testing to incorporate

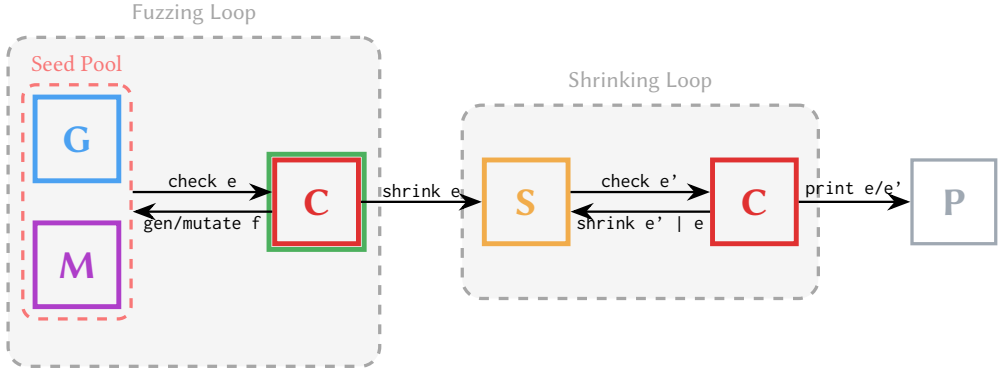


Fig. 5. An Abstract Representation of the FuzzChick Property Runner

fuzzing, FuzzChick reused AFL’s choices [Lampropoulos et al. 2019] instead of experimenting with new options in the new higher-level setting; in part because each option in such an experimentation would effectively require a new framework. Situations like this are precisely what a programmable and flexible runner framework addresses; in § 6.3.1, we carry out an extensive case study to explore what effect different search strategies and seed pool experimentations have in testing.

2.4 Custom-Feedback Guided (Targeted) Property Runner

Coverage information is not the only form of feedback that can be used to guide input generation. The literature is rife with domain-specific metrics that can be used to produce interesting inputs: in SlowFuzz [Petsios et al. 2017] and PerFFuzz [Lemieux et al. 2018], the goal of the testing campaign is to discover algorithmic complexity vulnerabilities that could lead to Denial of Service attacks in systems, which is achieved by maximizing instruction counts and execution path lengths; in SQLancer [SQLancer contributors 2022], the goal is to find bugs in SQL engines, achieved by maximizing the diversity of the generated query plans [Ba and Rigger 2023]. It is also possible to consolidate such different feedback forms under a single compositional design, as demonstrated by FuzzFactory [Padhye et al. 2019d] and Target [Löschner and Sagonas 2017], which allows users to define and combine different feedback functions for guiding the generation of inputs. Fig. 6 depicts an abstract representation of this last targeted property runner, where the feedback is a separate function that is computed over the input instead of a value obtained via instrumentation.

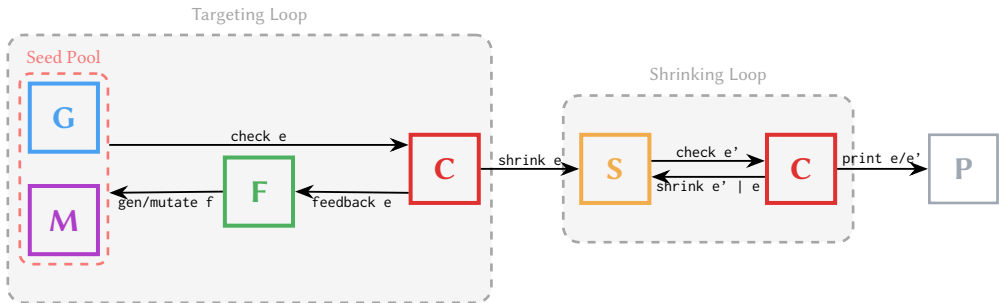


Fig. 6. An Abstract Representation of the Targeted Property Runner

Once again, however, implementing such runners almost always meant implementing an entirely new framework. Using our approach, implementing and experimenting with different forms of feedback is available to users at the user-space of the library without needing to dive into library internals.

2.5 Combinatorial Property Runner

There are yet different forms of feedback that can be useful in testing, that do not rely on dynamic feedback obtained during the execution of a test, but rather on static features of generated inputs. Combinatorial coverage offers one such alternative [Goldstein et al. 2021; Mittelman et al. 2023] by performing *online generator thinning*, generating several inputs at each iteration of the generate-and-check loop and only using the input that covers the maximum amount of constructor interactions. Such an approach, depicted in Fig. 7, is especially useful when executing a test can be costly (e.g. when testing compilers), and it can also be encoded using our approach.

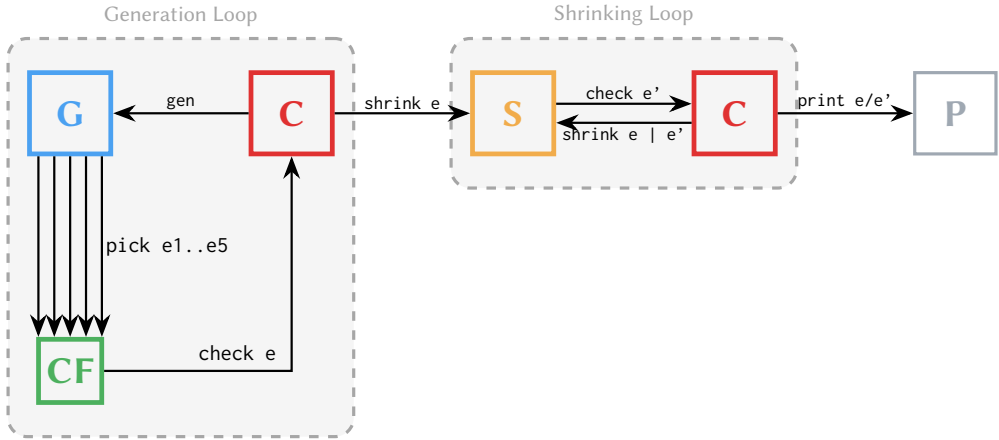


Fig. 7. An Abstract Representation of the Combinatorial Property Runner

2.6 Parallel Property Runner

Whereas the runners we explored so far focused on obtaining a better distribution of inputs by guiding the generator towards "interesting" parts of the search space, QuickerCheck [Krook et al. 2024] improved upon the standard loop by exploiting parallelism: letting users run more tests during the same time span. In earlier work, parallelism was utilized effectively by building a "swarm" [Groce et al. 2012] of testing campaigns, each member focusing on a different subset of operations or capabilities of the underlying system. QuickerCheck instead builds a parallel runner where each thread cooperatively works within the same campaign. Running independent QuickCheck campaigns naively can cause each worker to repeatedly explore the same small-sized part of the input space. QuickerCheck avoids this by sharing a growing size counter across workers.

In such a parallel runner, depicted in Fig. 8, different threads share a common atomic size counter for ensuring parallel progress on each thread while keeping the contention on shared resources to a minimum. The paper presents near-linear speedup for up to 6 threads due to the ability of almost perfect responsibility sharing across threads without contention. In § 6.5, we discuss a re-implementation of that runner on top of a Racket PBT library.

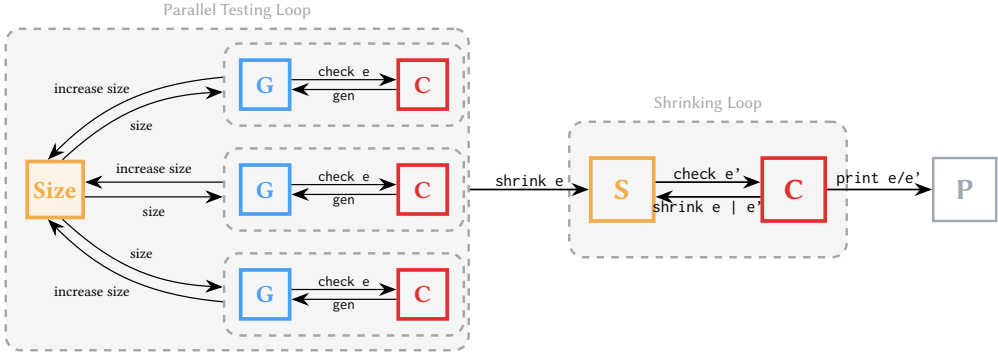


Fig. 8. An Abstract Representation of the QuickerCheck Property Runner

2.7 Discussion

The property runners we discussed in this section have given rise to a wide variety of frameworks across languages. Unfortunately, while their pictorial representations appear similar, their implementations are anything but: a tight coupling between property runners and property DSLs means that implementation details often render it very difficult, if not entirely impossible, to switch and experiment with different approaches without major changes to the underlying code. We discuss exactly how in the next section, before showcasing the flexibility provided by our framework by implementing *all* of the runners presented above in § 6.

3 Property Languages

Properties are simply *universally quantified predicates*. We want to define a layer of such properties on top of some standard host language, equipped with at least booleans:

$$\begin{aligned}\tau &:= \text{Bool} \mid \tau \rightarrow \tau \\ e &:= x \mid \lambda x. e \mid e \ e \mid T \mid F \mid \dots\end{aligned}$$

In practice, the host language needs to also support some kind of randomness for generation, lists for shrinking, strings for pretty printing, etc. For presentation purposes, let's begin by formally describing the core boolean structure:

$$p := \forall x : \tau. p \mid e$$

That is, properties are either universal quantifiers or an injection of a host predicate.

The core question we ask in this paper is: *how should we represent this language?* We'll start by reviewing existing embedding solutions from the literature, demonstrating why they are not flexible enough to allow for users to customize or specify their runners, before discussing our proposed solution in detail.

3.1 Background Terminology: Deep and Shallow Embeddings

An *embedding* involves two languages: an *object language* that is being implemented, and a *host language* that the object language is being implemented in. In a deep embedding, the terms of the object language are represented as inductive data in the host language. These representations allow for arbitrarily manipulating and inspecting the terms of the object language via the usual mechanism of pattern matching. By contrast, shallow embeddings directly expand the constructs of the object language in terms of constructs of the host language, which allows for straightforward implementation but no user-level term manipulation [Boulton et al. 1992]. Naturally, multiple

hybrid alternatives have been explored, trading off the simplicity of implementation of shallow embeddings with the flexibility of deep ones [Chlipala 2008; Li and Weirich 2022; Prinz et al. 2022].

3.2 Status Quo: A Shallow Property Language

Defining a property language requires a construct for defining its inputs via universal quantification ($\forall$), a construct for defining the boolean predicate that is validated through the property (*check*), and finally a mechanism for running the property. In PBT libraries using a shallow embedding following QuickCheck’s design, the mechanism is using host language lambdas as binders for the universal quantification.

$$\begin{aligned} \text{forall} & : \forall \tau. (\tau \rightarrow \text{prop}) \rightarrow \text{prop} \\ \text{check} & : \text{Bool} \rightarrow \text{prop} \\ \text{run} & : \text{prop} \rightarrow \text{IO } () \end{aligned}$$

Concretely, the type of properties is some type *prop* in the host language (often restricted via typeclass-like mechanism), *check* injects a host boolean into this type, and *forall* takes a host-level function and turns it into a property. Finally, frameworks also provide a *run* function to test properties constructed using this API.

The main advantage of this approach is that it is very convenient for users to write properties (using host-level binders), and extremely easy to test them (simply invoking the framework-provided *run*). Implementing such a framework as a developer is also straightforward— a minimal implementation of this shallow API in Haskell using splittable pseudorandomness [Claessen and Palka 2013] is shown in Fig. 9: the type of generators for some type *a* is simply a wrapper around a function from some source of randomness to *a*; properties are generators for *Results* (a datatype that encodes at least whether testing succeeded); *forall* takes a generator and a predicate and binds them together taking care of the underlying randomness; and the *run* loop simply generates and checks a result up to some predefined limit on the number of tests, taking care again of the underlying randomness, and reporting success or failure in the end.

```
newtype Gen a = Gen (StdGen -> a)

type Property = Gen Result
data Result = OK | Failed

forall :: Gen a -> (a -> Property) -> Property
forall (Gen g) f = Gen (\r -> let (r1,r2) = split r
                               Gen h = f (g r1)
                               in h r2)

run :: Property -> IO ()
run (Gen g) = do newStdGen >>= loop numTests
  where loop 0 _ = ... -- report success
loop n r = let (r1, r2) = split r in case g r1 of
  OK      -> loop (n-1) r2
  Failed -> ... -- report counterexample
```

Fig. 9. An implementation of QuickCheck’s core functionality.

In practice, implementations of such frameworks are more involved to account for better reporting, minimizing counterexamples, etc. However, a key characteristic of this approach is that the `Property` type is opaque to users: it's simply a wrapper around a function. As a result, users cannot inspect its structure or customize how properties are tested without modifying the definition of `Property` and the internals of the framework.

3.3 Hypothetical: A Deep Property Language

The polar opposite of the current shallow embedded paradigm would be to use a deep embedding: define an inductive representation of both the host language and the language of properties and require users to program properties in that. That would mean that users of the testing framework would have to learn *yet another* language in which to write their specifications, foregoing all of the convenience that host-language binders provide. Unsurprisingly, no frameworks have taken this route: it's hard enough to convince users to write properties and specifications without additional burdens to adoption [Goldstein et al. 2024].

3.4 Proposal: A Mixed Property Language

With pure shallow and deep embeddings being unsatisfactory, we turn to hybrid approaches to attempt to regain some amount of inspection and manipulation capabilities while keeping the property-writing process as ergonomic as possible. Researchers have explored a wide range of *mixed embeddings* that reify parts of the language being considered. For example, we could attempt a HOAS-style approach, where we define an inductive type of properties, with a constructor corresponding to *forall* and *check*, both indexed by the arguments of their shallow counterparts:

$$\begin{aligned} \text{Inductive Prop} &:= \\ &| \text{Forall} : (\tau \rightarrow \text{Prop}) \rightarrow \text{Prop} \\ &| \text{Check} : \text{Bool} \rightarrow \text{Prop} \end{aligned}$$

Such a representation still allows us to encode the universal quantifiers using host language binders, but also makes some headway into the issue at hand—given a property we can now pattern match against it! However, there is still a problem: to access the “rest” of the property that follows a universal quantification, we need an element of the type being quantified over; since such elements are going to be randomly generated, we can't actually recurse down the structure of the property without restricting ourselves to something along the lines of QuickCheck's Generator monad.

The key issue is that we're hiding the definition of the “rest” of the property under a host-language binder: the *Forall* constructor takes a function as an argument, which we cannot pattern match on to go deeper. Could we use standard host language constructs to define the predicates at the leaves of the property (the *Checks*), while retaining the ability to pattern match on the property structure?

Proposal: Deferred Binding Abstract Syntax. Our solution is what we call *deferred binding abstract syntax* (DBAS): rather than binding a variable once at the site of its universal quantification, we will instead bind it at every one of its use sites.

$$\begin{aligned} \text{Inductive Prop (env : [Type])} &:= \\ &| \text{Forall} : \forall \tau. \text{Prop } (\tau :: \text{env}) \rightarrow \text{Prop env} \\ &| \text{Check} : (\llbracket \text{env} \rrbracket \rightarrow \text{Bool}) \rightarrow \text{Prop env} \end{aligned}$$

We still define an inductive type of properties, with one constructor for each combinator in our API. We also index our type of properties by an environment: a list of types that have already been quantified. The key change is that we move the host level binder from the binding site (*Forall*) to its use site (the *Check*). That is, the argument to *Forall* is no longer a function, but simply a property

with an extended environment; on the other hand, the argument to *Check* is no longer a simple boolean, but a function that binds *everything* in the environment. In the code above, we denote that as $\llbracket env \rrbracket$ —we will see how it can be implemented in a statically typed or dynamically typed setting respectively in later sections.

At first glance, this is a counter-intuitive trade-off: every time you want to use a variable, you have to bind *everything* that was quantified before that point. That would only make sense in a scenario where there are a lot of quantifications and few variable uses—which is precisely the case for the language of properties! Compared to the previous language representations, this DBAS-based one allows us to access the structure of the “rest” of the predicate without having access to a concrete value, which is necessary when such values are to be randomly generated. Finally, compared to the fully shallow representation, the added inductive structure and typing information pose some burden to the user experience, but once again, we’ll address these issues in host-language-specific settings.

Adding Annotations. Generalizing the property language above to include annotations for generation, shrinking, or printing of individual elements is straightforward. We simply include an optional extensible list of annotations at the *Forall* constructor:

$$\begin{aligned} \text{Inductive Prop } (env : [Type]) := \\ &| \text{Forall} : \forall \tau. as \rightarrow Prop (\tau :: env) \rightarrow Prop env \\ &| \text{Check} : (\llbracket env \rrbracket \rightarrow Bool) \rightarrow Prop env \\ \\ as &:= \emptyset \mid (k, \forall \tau. \llbracket env \rrbracket \rightarrow \tau) :: as \\ k &:= gen \mid shr \mid \dots \end{aligned}$$

At a high level, we can annotate each *Forall* constructor with a (possibly empty) sequence of (host-level) functions that quantify over the context so far (as in *Check*) and return annotation-specific terms (e.g. a generator or a shrinker). We demonstrate the exact implementation of such annotations in the following two host-language-specific sections.

4 A Dependently Typed Property Language

Implementing the language of universally quantified properties using deferred binding abstract syntax is straightforward on top of a dependently typed language, but achieving good ergonomics can be a challenge. In this section we will focus on implementing such a language on top of the QuickChick [Lampropoulos 2018] framework for property-based testing in Rocq.

To that end, we explicitly encode contexts in our properties, capturing every input that *will have been generated* by that point. We will use a standard inductive definition of contexts, using $\emptyset$ to denote the empty context and $\cdot$ to extend a context by a type. Given a context, we can calculate the type corresponding to it: the type of tuples containing all of its types in order, with unit as the base case:

```
Inductive Ctx :=
  |  $\emptyset$  : Ctx
  |  $\cdot$  : Type  $\rightarrow$  Ctx  $\rightarrow$  Ctx.

Fixpoint interp (C : Ctx) : Type :=
  match C with
  |  $\emptyset$  => unit
  | T  $\cdot$  C => T * interp C
  end.
```

We will write $\llbracket C \rrbracket$ as a shorthand for `interp C`.

Now we can define a deeper version of the property language using DBAS:

```
Inductive Prop : Ctx → Type :=
| FORALL : forall {A: Type} {C: Ctx} (name: string)
  (generator :  $\llbracket C \rrbracket \rightarrow G\ A$ ) (mutator :  $\llbracket C \rrbracket \rightarrow A \rightarrow G\ A$ )
  (shrinker :  $\llbracket C \rrbracket \rightarrow A \rightarrow \text{list } A$ ) (printer :  $\llbracket C \rrbracket \rightarrow A \rightarrow \text{string}$ ),
  Prop (A · C) → Prop C
| IMPLIES : forall C
  (prop :  $\llbracket C \rrbracket \rightarrow \text{bool}$ ),
  Prop C → Prop C
| CHECK : forall C,
  ( $\llbracket C \rrbracket \rightarrow \text{bool}$ ) → Prop C.
```

Just like the shallow approach of QuickCheck, this representation allows us to express, in the host language, type-based generators, mutators, shrinkers, and printers for each quantifier in a property. Just like the shallow approach, we can use typeclasses to automate much of the burden of specifying the property (as we will see below). Crucially, however, unlike the shallow approach we can pattern match on this definition and construct a wide range of methods for interpreting such properties without needing to modify the code of the underlying property-based framework at all.

For example, the standard “generate-and-run” loop of Figure 1, which amounts to interpreting such a property in the original shallow embedding, can be straightforwardly encoded as follows, first defining a simple type of *results* that holds information such as the inputs that were generated (calculated recursively from an input property prop):

```
Inductive RunResult {C: Ctx} (prop : Prop C) :=
| Normal :  $\llbracket \text{inputs prop} \rrbracket \rightarrow \text{bool} \rightarrow \text{RunResult prop}$ 
| Discard :  $\llbracket \text{inputs prop} \rrbracket \rightarrow \text{RunResult prop}$ .
```

And then the runner as a straightforward fixpoint as shown in Fig. 10¹.

The flexibility to define such a loop at the hands of users allows for encoding all kinds of interpreters for properties, including pure generators, runners, shrinkers, fuzzers, with fully programmable execution, printing, and benchmarking options. We’ll further demonstrate this flexibility by implementing a series of runners from the literature, all on top of this abstract property language.

Usable Defaults with Dependently Typed Programming. A standard disadvantage of deep embeddings compared to shallow ones, is that they are generally less convenient to work with. Encoding everything in the host language, as in a shallow embedding, allows users to simply reuse a large part of host language infrastructure. The property language described above enables much of that using dependent types. Still, it is desirable to provide as seamless an experience for new users as possible, leveraging the same familiar typeclass-based interfaces of the shallow setting.

For concreteness, without any effort to provide such an experience, users would have to write the following to encode the optimization correctness property of the introduction:

```
Definition prop_eval_bad :=
  FORALL (fun tt => gen) (fun tt => mut)
    (fun tt => shrink) (fun tt => pretty) (
    @CHECK (Expr ·  $\emptyset$ ) (fun '(e, _) => eval e == eval (optimize e))).
```

¹Branches follow the standard [convoy pattern](#) to enable type inference in dependent pattern matching, and we hide those in *gray*—they are unfortunate artifacts of Rocq’s support for dependently typed programming.

```

Fixpoint genAndRun (C : Ctx) (prop : Prop C) :  $\llbracket C \rrbracket \rightarrow G$  (RunResult prop) :=
  match prop with
  | FORALL A C name gen mut shr pri prop =>
    (fun A' C' name' gen' mut' shr' pri' prop' =>
      fun env =>
        a ← gen' env;;
        res ← genAndRun (A' · C') prop' (a, env);;
        match res with
        | Normal env truth => (fun env' truth' =>
          ret (Normal (Some a, env') truth')) env truth
        | Discard env => (fun env' =>
          ret (Discard (Some a, env')) env
        end) A C name gen mut shr pri prop
    | CHECK C prop => (fun C' prop' =>
      fun env =>
        ret (Normal tt (prop' env))) C prop
    | IMPLIES C pre prop => (fun C' pre' prop' =>
      fun env =>
        if pre' env then
          res ← genAndRun C' prop' env;;
          match res with
          | Normal env truth => (fun env' truth' =>
            ret (Normal env' truth')) env truth
          | Discard env => (fun env' =>
            ret (Discard env') env
          end)
        else ret (Discard (nones prop')) C pre prop
      end.
    
```

Fig. 10. Implementation of a DBAS-powered generate-and-run loop in Rocq

That is, users would have to write a lot of annotations to achieve the same result, both at the type level ($\text{Expr} \cdot \emptyset$) and to annotate individual **FORALLS** with the various generators, shrinkers, and printers.

However, we are not restricted to providing the core property definition as the final user-level interface. Instead, we develop a simple surface-level language that allows users to write simplified properties, using typeclasses to fill in the remaining information. For example, the same property can be defined in our framework as in the much more straightforward snippet that follows:

```

Definition prop_eval :=
  ForAll e :- Expr,
  Check (fun 'e => eval e == eval (optimize e)).
  
```

In addition, users can override particular aspects of the property with lightweight annotations. For example, specifying a particular generator to be used, such as `gen` can be done as follows:

```

Definition prop_eval :=
  ForAll e :- Expr gen:gen,
  Check (fun 'e => eval e == eval (optimize e)).
  
```

Most of this surface language is achieved using Rocq’s powerful notation mechanism, including its support for recursive notations. The final piece of the puzzle to simplify `Check` definitions relies on typeclasses. In particular, we associate each predicate with its corresponding context and a proof of that correspondence, in a class we name `Untuple`:

```
Class Untuple (A : Type) :=
{ untuple : Ctx
; untuple_correct :  $\llbracket \text{untuple} \rrbracket = A$  }.
```

We then provide instances for the empty context and the bind:

```
Instance Untuple_empty : Untuple unit :=
{ untuple :=  $\emptyset$ 
; untuple_correct := eq_refl }.

Instance Untuple_pair {A B} {Untuple B} : Untuple (A * B) :=
{ untuple := A ..untuple B _
; untuple_correct := ... }.
```

Before finally providing a convenient user-level wrapper for property conclusions, which we used above:

```
Definition Check {A} {Untuple A} (p : A  $\rightarrow$  bool) : Prop (·untuple A _).
  refine (Check (·untuple A _) _).
  rewrite untuple_correct.
  exact p.
Defined.
```

Finally, we can also leverage the extensive metaprogramming facilities of QuickChick to construct a property definition directly from a predicate:

```
Definition eval_correct (e : Expr) := eval e == eval (optimize e).
Derive Property eval_correct.
(* ==> eval_correct_prop is defined. *)
```

This command constructs the deeper property above using the Rocq predicate itself.

5 A Dynamically Typed Property Language

Representing the property language with deferred binding abstract syntax is not restricted to a dependently typed setting; in this section, we show how to implement it in a dynamically typed language like Racket. In such a setting, we no longer have static guarantees about the types of the variables in the context; instead, we fall back on dynamic errors. However, we are also not encumbered by the type system, as we can freely invoke functions on arguments of (statically) unknown types, which we will fully take advantage of to recover most of the convenience of a shallow representation.

The first step is to directly translate the datatype into a series of `structs`:

```
(struct Forall (var augments body))
(struct Implies (prop body))
(struct Check (prop))
```

In Racket, we cannot rely on typeclasses to automatically discover generators or shrinkers for property-defined variables. Instead, we add a dictionary to the `Foralls` that allows us to attach extra information onto each variable that we call *augments*. Concretely, this dictionary maps augment names to functions that take the current environment of previously-generated variables and produce the augment value. We write augment names as Racket keywords: identifiers prefixed

with `#:` that serve as named dictionary keys. These augments are fully generic in that they can store any values, though our implementation defines specific uses for three.

- `#:contract` attaches an invariant contract: a predicate checked dynamically against each generated value bound to the variable.
- `#:gen` attaches a generator for the variable. In order to make the usage of generators from other frameworks e.g. `RACKCHECK` easier, we intentionally treat the generator value as opaque. Instead, property interpretations that use the generators take a user-provided sampling function that is applied to the generator.
- `#:shrink` attaches a function used for shrinking counterexamples.

However, handling the struct-based definitions directly involves a lot of explicit plumbing that we would rather not need to write. Consider once again the optimization correctness property:

```
(define eval-opt
  (Forall 'e (hash '#:contract (λ (env) expr?) '#:gen (λ (env) gen-expr))
    (Check (λ (env)
      (let ([e (dict-ref env 'e)])
        (equal? (eval e) (eval (optimize e))))))))
```

There are two main ergonomic issues: the repeated nesting and the explicit environment passing and lookup. We can use Racket's extensive macro capabilities to create a DSL for writing these deeper properties. To that end, we flatten the nested structure by using the fact that properties are isomorphic to a list of `Forall` and `Implies` terminated by a single `Check`. Then, we use Racket's variable transformer macros to define and refer to generator variables as Racket identifiers and insert the dictionary passing plumbing for us.

These features allow us to translate the above roundtrip property into one much closer to how shallow embedding properties are written.

```
(define eval-opt
  (property
    (forall e #:contract expr? #:gen gen-expr)
    (equal? (eval e) (eval (optimize e)))))
```

Property Runners. Developing property runners in the Racket setting shares much of the structure of the Rocq version, with some extra logic to attach the contract to the generated value if present (Fig. 11). Encoding properties using deferred binding abstract syntax in Racket gives users the same variety in choices of property runners loops that the Rocq version does, as well as the same programmability for expert users. We utilize Racket's contracts to optionally allow users to enforce typed boundaries on generated variables. Racket's powerful macros enable us to write properties in a style that requires little syntactic overhead compared to shallow embeddings without sacrificing any of the programmability provided by deeper embeddings. In the next section, we show that the extra programmability does not come at a performance cost, enables the writing of property runners that execute many runs in parallel, and shrinking loops that find significantly smaller counterexamples.

6 Evaluation

In this section we evaluate our approach and implementation:

- (1) First, we demonstrate the expressive power of DBAS, showing that it allows for building new property runners without any dependence or modifications to the library internals: we implemented all of the property runners we presented in § 2. Here, we discuss two such

```

(define (gen-and-run p sample . args)
  (let loop ([p p] [env (hash)])
    (match p
      [(Forall var augments body)
       ; Ensure the variable has a generator augment
       (unless (dict-has-key? augments '#:gen) (error 'no-generator))
       ; Generate a value using the sample function
       (define val (apply sample ((dict-ref augments '#:gen) env) args))
       ; Check the contract if present
       (when (dict-has-key? augments '#:contract)
         (invariant-assertion ((dict-ref augments '#:contract) env) val))
       ; Recur
       (loop body (dict-set env var val)))]
      [(Implies prop body)
       (if (prop env) ; Check precondition
           (loop body env) ; If it passes, recur
           (values 'discard env))] ; If it fails, discard
      [(Check prop)
       (if (prop env) ; Check result
           (values 'pass env) ; Success
           (values 'fail env)))])) ; Failure

```

Fig. 11. Implementation of a DBAS-powered generate-and-run loop in Racket

runners and their implementations in detail, the standard QuickCheck-inspired property runner (§ 2.1) and the mutation-based coverage-guided runner (§ 2.3). We provide the rest of the implementations as supplementary material in Appendix A.

- (2) Second, we evaluate the performance overhead of DBAS-based implementations, showing that it is comparable to their shallowly embedded counterparts. In § 6.2 we compare the performance of the DBAS-based property runner as implemented in Rocq and Racket to the existing property runners of QUICKCHICK and RACKCHECK. We found that using the DBAS-based embedding has *no observable performance overhead*.
- (3) Finally, we showcase how the added expressivity allows these experiments to be expressed as changes to reusable runners and runner components, rather than as changes to the property language or framework internals. We carry out three experiments: we explore how different design choices with respect to the representation and sampling of the seed pool affect testing performance, improving FUZZCHICK in the process (§ 6.3.1); we compare the default integrated shrinking capabilities of RACKCHECK with a simple external shrinker we implemented for a DBAS-style Racket library (§ 6.4); and we implement and benchmark a parallel property-runner inspired by a recent work on parallelizing QuickCheck [Krook et al. 2024] (§ 6.5).

6.1 Property Runners with DBAS

We demonstrate the expressiveness of DBAS by implementing all property runners from § 2. In the last two sections we saw how to implement a simple `genAndRun` loop in both Rocq and Racket. Leveraging the ability to pattern match on properties and recurse down their structure, we can similarly develop building blocks that will help implement all property runners. In particular, for

this section, we'll use a **generator**, which produces random inputs for a property; a **mutator**, which given inputs to a property mutates some or all of them; a **runner**, which given inputs to a property executes it; a **shrinker**, which given a way to minimize a property's inputs and a counterexample finds a locally minimal counterexample using best-first search; and a **printer**, for reporting counterexamples. The underlying pattern matching ability is still useful in the presence of these abstractions. For example, we will also write domain-specific versions of these functions for some of our evaluation, such as an **instrumentedRunner** that runs a property collecting instrumentation feedback in the process.

6.1.1 QuickCheck-Style Property Runner. Using the components outlined above we can implement any variation of the standard generational runner, with code that closely follows its depiction. Fig. 12 shows the implementation of the basic generate-and-test then shrink-and-test property runner. Fig. 13 shows the same runner in Racket, with minor syntactical adjustments.

```
Definition runLoop (fuel : nat) (cprop : Prop 0) :=
  let fix runLoop' (fuel : nat) (cprop : Prop 0)
    (passed : nat) (discards : nat) : G Result :=
    match fuel with
    | 0 => ret (mkResult discards false passed [])
    | S fuel' =>
      input <- gen cprop (log2 (passed + discards));;
      res <- run cprop input ;;
      match res with
      | Normal seed false => (* Fails *)
        let shrunk := shrinker 10 cprop seed in
        let printed := print cprop 0 shrunk in
        ret (mkResult discards true (passed + 1) printed)
      | Normal _ true => (* Passes *)
        runLoop' fuel' cprop (passed + 1) discards
      | Discard _ _ => (* Discard *)
        runLoop' fuel' cprop passed (discards + 1)
    end
  end in
runLoop' fuel cprop 0 0.
```

Fig. 12. Simple Generational Property Runner in Rocq

Each component of the property runner in Fig. 1 has a clear correspondence to the corresponding overlaid sections in Fig. 12. The functions **gen**, **run**, **shrinker**, and **print** are the building blocks of user level property-runners, but can also be written by users themselves in a straightforward manner as shown in Sections 4 and 5. We use them here to present runners at a higher level of abstraction as enabled by DBAS, focusing on how these components interact with each other in order to show-case users how to implement new runners corresponding to their needs.

The runner itself consists of two tight loops, the first one running **gen**, **run**, **gen**, **run**... until a counterexample is found, or until a predefined limit of tests has been reached. The second loop runs **shrink**, **run**, **shrink**, **run**... until it is not able to minimize the input further, reporting the smallest input within the shrinking process.

```
(define (run-loop tests p)
  (let loop ([n 0] [passed 0] [discards 0])
    (if (= n tests) (result #f passed discards #f)
      (let ([env (generate p run-rackcheck-gen (floor (log n 2)))] )
        (case (check-property p env)
          [(fail) (result #t passed discards (shrink-eager p env))]
          [(pass) (loop (add1 n) (add1 passed) discards)]
          [(discard) (loop (add1 n) passed (add1 discards))])))
```

Fig. 13. Simple Generational Property Runner in Racket

As users now have access to the building blocks for writing this loop, they are empowered to change it! For instance, we have experimented with implementing a precondition bypassing `run` function that can be used to optimize property execution for correct-by-construction generators by skipping the preconditions. The users can change the shrinking algorithm, adjust sizes, move to structured printing, or even write the results directly to a file. They can decide not to terminate the test after a single counterexample, add specific budgets for number of discards or time bounds if they decide to. All of these choices, and many more that we envision users will invent, can be used to augment the effectiveness of testing.

```

Definition fuzzLoop (fuel : nat) (cprop : Prop 0) {Pool} (seeds : Pool) : G Result :=
  let fix fuzzLoop' (fuel passed discards: nat) seeds := match fuel with
  | 0 => ret (mkResult discards false passed [])
  | S fuel' =>
    let directive := sample seeds in
    input ← match directive with
    | Generate => gen cprop (log2 (passed + discards))
    | Mutate source => mutate cprop source
    end;;
    res <- instrumentedRun cprop withInstrumentation;;
    let '(res, feedback) := res in
    match res with
    | Normal seed false => (* Fails *)
      let shrunk := shrinkLoop 10 cprop seed in
      let printed := print cprop 0 shrunk in
      ret (mkResult discards true (passed + 1) printed)
    | Normal seed true => (* Passes *) match useful seeds feedback with
    | true =>
      let seeds' := invest (seed, feedback) seeds in
      fuzzLoop' fuel' (passed + 1) discards seeds'
    | false =>
      let seeds' := match directive with
      | Generate => seeds
      | Mutate _ => revise seeds
      end in fuzzLoop' fuel' (passed + 1) discards seeds'
    end
  | Discard _ _ => (* Discards *) match directive with
  | Generate => fuzzLoop' fuel' passed (discards+1) seeds
  | Mutate source => match useful seeds feedback with
  | true => fuzzLoop' fuel' passed (discards+1) seeds
  | false => fuzzLoop' fuel' passed (discards+1) (revise seeds)
  end
  end
end
end in fuzzLoop' fuel 0 0 seeds.

```

Fig. 14. Coverage-Guided Fuzzing Property Runner in Rocq

6.1.2 Mutation-Based Property Runners. Property-Based Testing, for a long time, has relied solely on random generation from scratch as opposed to mutating existing inputs as fuzzing tools do. This design has been challenged in Targeted PBT [Löschner and Sagonas 2017] with a mutational approach that optimizes explicit user-provided feedback functions, and later by FuzzChick [Lampropoulos et al. 2019] and Zest [Padhye et al. 2019b], which rely on branch coverage information obtained via binary instrumentation, followed by libraries such as HypoFuzz [Hatfield-Dodds and contributors 2025] and Bolero [Cameron Bytheway 2019].

The coverage-guided fuzzing property runner introduces some complexity on top of the simple QuickCheck-style runner. This complexity is mainly related to the orchestration logic that manages the seed pool, which is parametric over the `SeedPool` interface. At each iteration of the loop, the seed pool produces a directive, either to generate an input from scratch, or mutate a previous input. The generated input is then passed into `instrumentedRun` function that is also parameterized over a custom instrumentation function.

In classic fuzz testing, this instrumentation function is branch or path coverage, yet our Rocq library can accommodate any function that observes information about the state of the executed program, as in Padhye et al. [2019d]. This is reflected in the fuzzing loop in Fig. 14 where feedback is received from the execution of the `instrumentedRun`. Such feedback can range from traditional branch or path coverage (as in coverage-guided fuzzing) to timing or memory usage (as in performance fuzzing [Lemieux et al. 2018]).

This way, we view the fuzzing property runner presented in Fig. 14 as (1) a more generalized version of the classic coverage-guided fuzzing, and (2) a property-based testing version of FuzzFactory [Padhye et al. 2019d], which allowed for arbitrary instrumentation functions to guide the search similar to the fuzzing property runner we present here.

An important design decision we must make when implementing a DBAS property encoding in a statically typed language, such as our Rocq encoding, is which associated functions are part of the property encoding. These recent advances in mutational PBT have led us to make mutation a first class citizen of our library, and we have implemented a generic seed pool interface that mutation-based property runners can leverage. The seed pool interface abstracts away search strategy (how to select which input to mutate) and power schedule (how long to fuzz it for) concerns. We will revisit this abstraction in just a few sections (Sec 6.3.1). We have implemented two property runners using these components: a coverage-guided fuzzing property runner as presented in Fig. 5, and a custom feedback-guided (targeted) property runner as presented in Fig. 6. We present the code of the fuzzing property runner in detail in Fig. 14, and leave the presentation of the targeted property runner to the Appendix.

6.2 Comparison of Deep and Shallow Embeddings

In this section, we compare the performance of a DBAS-powered implementation to its shallow counterpart. We turn to the ETNA [Shi et al. 2023] framework for evaluating property-based testing performance, which comes with a series of Rocq workloads—programs along with injected bugs—in the form of Binary Search Trees, Red-Black Trees, and the Simply Typed Lambda Calculus. For the purposes of this experiment we extended the ETNA tool to support Racket and ported these workloads.

Throughout the sequence of case studies that follow, our performance results will use ETNA bucket charts: each bucket represents the tasks (mutant-property pairs) solved within a certain time limit in the average of 10 trial runs. The leftmost bucket with the darkest color denotes the tasks solved within 0.1 seconds, where the remaining buckets progressively denote the tasks solved



Fig. 15. ETNA Style Bucket Chart Legend

within 1, 10, 60 seconds, and the last bucket denotes the tasks that were not solved within 60 seconds for at least one of the 10 trial runs. The legend for these charts is shown in Fig. 15.

6.2.1 Comparison of Deep and Shallow Embeddings in Rocq. Our first case study focuses on the performance implications of using deferred binding abstract syntax instead of the standard QuickCheck-style runner.

We benchmark our implementations of this loop for both Rocq and Racket against the existing loops of the QUICKCHICK and RACKCHECK libraries on 3 ETNA workloads: binary search trees (BST), red-black trees (RBT), and the simply-typed lambda calculus (STLC). Our results show that libraries implemented via DBAS, using the runner we showed earlier in this section, is on par with both QUICKCHICK and RACKCHECK.

The bucket charts in Fig. 16 show that using DBAS does not incur a performance penalty compared to QUICKCHICK in the BST, RBT, and STLC workloads. In a total of 12 strategy/workload pairs, DBAS-style Rocq library outperforms QUICKCHICK in 9 of them, while QUICKCHICK has a better performance in 3 of them. Yet, there are no notable differences in the results of the two libraries in terms of mean time to solve the tasks.

6.2.2 Comparison of Deep and Shallow Embeddings in Racket. In our Racket experiments, we focused on comparing the DBAS-style Racket library with the shallow embedding implemented by RACKCHECK. We have conducted our experiments on the same BST, RBT, and STLC workloads by porting the existing workloads in Haskell to Racket. In the process of porting these workloads, we have also discovered and reported a bug in the RACKCHECK core, which the authors have fixed in the latest version of the library [Keles 2024].

Fig. 17 shows the results of the comparison of the deep and shallow embeddings in Racket. Once again, we find no notable differences in performance between the two versions. However, during the course of these experiments we found that the default configuration of RACKCHECK in terms of size of generated terms led to considerable performance degradation in the RBT case study. As size is configurable in most PBT APIs, we have changed the RACKCHECK size function to be logarithmic with respect to number of tests, as our property-runner does. Still, this further reinforces our point on programmability: if sizes and similar aspects of generation are configurable (and severely impact testing performance), why shouldn't the runners themselves be configurable?

6.3 DBAS-powered Experiments

In this section, we carry out three case studies that the expressiveness of DBAS enabled us to.

6.3.1 An Exploration of Seed Pool Design Choices. First, we turn to feedback-guided property runners and their programmability. Fuzzing research often explores different strategies and power schedules [Böhme et al. 2020, 2019; Chen et al. 2023; Fioraldi et al. 2020], with researchers coming up with better and better designs. On the other hand, property-based testing frameworks that support feedback-guided generation of inputs such as FuzzChick or Hypothesis, pride themselves in the power of testing arbitrary user-defined specifications, but do not provide an option to configure such crucial parameters of their feedback-guided property runners. We illustrated such a feedback-guided runner earlier in this section, that abstracts away such concerns into a small API, which we implement in Rocq using typeclasses (Fig. 18).

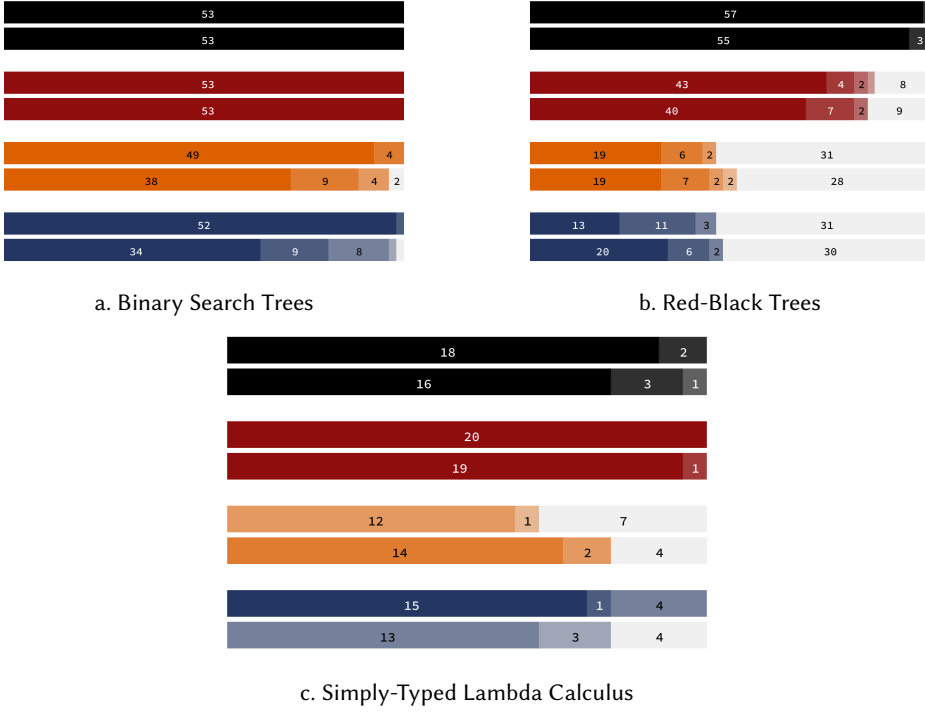


Fig. 16. Comparison of Shallow and Deep Embeddings in Rocq. Buckets follow the legend in Fig. 15. Each color denotes a strategy, where the top bar is DBAS and the bottom bar is the shallow behavior.

■ = Bespoke Generator, ■ = Specification-Based Generator, ■ = Type-Based Fuzzer, ■ = Type-Based Generator.

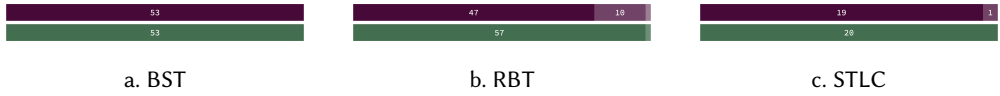


Fig. 17. Comparison of Shallow(RACKCHECK) and Deep(DBAS) Embeddings in Racket. Buckets follow the legend in Fig. 15. The purple bar on the top is DBAS with deep embedding, the green bar on the bottom is RACKCHECK with the shallow embedding.

■ = Bespoke Generator used with DBAS Library, ■ = Bespoke Generator used with RACKCHECK Library.

The configurability enabled by DBAS allows the users to rely on a set of community-accepted defaults chosen by framework developers, but also to explore if different choices from the literature or novel ones they devised fit their testing needs better. In this case study, we explore six different data structure representations to hold interesting seeds, as well as four different power schedules, leading to a total of 21 different configurations. We picked these configurations to explore different parts of the design space, such as the queuing strategy, the size/cardinality of the pool, whether the pool is monotonic, and how many times a given seed is reused. We have conducted our experiments on the IFC workload in ETNA, and we have used a type-based generator alongside a type-based mutator to conduct our experiments. While our experiment reveals a clear winner for this case

```

Class SeedPool {A F Pool: Type} := {
  (* Creates an empty pool. *)
  mkPool : unit → Pool;
  (* Adds a useful seed into the pool. *)
  invest : (A * F) → Pool → Pool;
  (* Decreases the energy of a seed after
  a useless trial. *)
  revise : Pool → Pool;
  (* Samples the pool for an input. *)
  sample : Pool → Directive A F;
  (* Returns the best seed in the pool. *)
  best : Pool → option (Seed A F);
}.

Class Utility {A F Pool: Type}
  {SeedPool A F Pool} := {
  (*
  Returns true if the feedback
  is interesting.
  *)
  useful : Pool → F → bool;
  (*
  Returns a metric of how interesting
  the feedback is.
  *)
  utility : Pool → F → Z;
}.

```

Fig. 18. SeedPool and Utility typeclasses used in Feedback-Guided Property Runners

study, we reiterate that our primary goal is not the exploration itself, but rather to demonstrate that such an exploration can be expressed by varying reusable runner components without modifying the property language or framework internals.

More concretely, we explore the following data structure representations, three that hold collections of seeds (as in FuzzChick), and three that only hold a single seed (as in Targeted PBT):

- (1) *FIFO Queue Seed Pool*: A pool that holds a queue of seeds, and reduces the energy of the current seed after usage. The pool only generates a new seed from scratch when the queue is empty, and mutates the seed otherwise. When the energy of the current seed drops to 0, it is removed from the queue. The next seed is chosen from the front of the queue. This was the default behavior of FuzzChick.
- (2) *FILO Queue Seed Pool*: The same as the *FIFO Queue Seed Pool*, but the next seed is chosen from the back of the queue.
- (3) *Heap Seed Pool*: Similar to the FIFO and FILO Queues, but the seeds are stored in a heap, creating a priority queue.
- (4) *Static Singleton Pool*: A pool that holds a single seed, and does not reduce its energy after usage. The pool generates a new seed from scratch at the first iteration, and mutates its seed for the subsequent iterations. The seed is only updated when a new seed with a better feedback is generated via mutation. This essentially devolves the search to hill climbing, as in the original Targeted PBT work [Löscher and Sagonas 2017].
- (5) *Dynamic Monotonic Singleton Pool*: A pool that holds a single seed, and reduces its energy after usage. The pool generates a new seed from scratch at the first iteration and when the energy of the current seed is 0, mutates the seed otherwise. The seed is only updated when a new seed with a better feedback is generated.
- (6) *Dynamic Resetting Singleton Pool*: A pool that holds a single seed, and reduces its energy after usage. As opposed to the *Dynamic Monotonic Singleton Pool*, once the current seed's energy is 0, the seed is effectively discarded and a new seed is generated from scratch.

All of the queues except the *Static Singleton Pool* have been tested with 4 different energy schedules, where the energy of a new seed was respectively up to 1, 10, 100, 1000, depending on its interestingness. We report the experiments over 5 trials for each configuration within 65 tasks in the IFC workload in ETNA. For brevity, the graphs omit 34 tasks none of the configurations have solved, and only show the remaining 31 tasks. Each bucket represents the tasks solved within a certain time limit for at least one of the 5 trials, where the leftmost bucket with the darkest color

denotes the tasks solved within 0.1 seconds, where the remaining buckets progressively denote the tasks solved within 1, 10, 60 seconds, and the last bucket denotes the tasks that were not solved within 60 seconds.

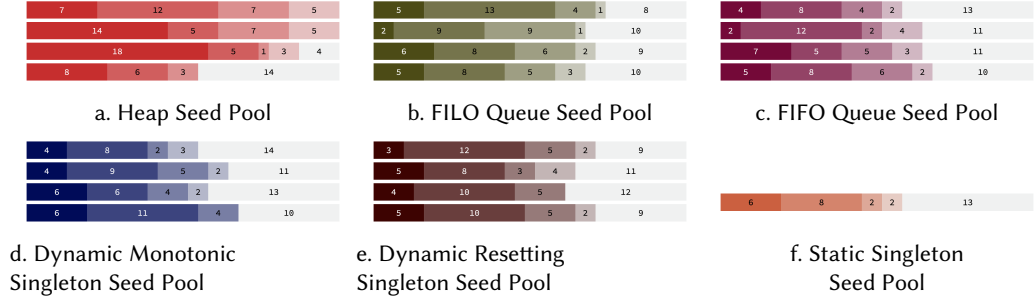


Fig. 19. Comparison of Seedpool Strategies in Rocq. Buckets follow the legend in Fig. 15. Each color denotes a strategy, where the bars are ordered from energy levels 1, 10, 100, and 1000; except for the static singleton pool that ignores energy.

These results mark a task as “solved” for the purposes of a bucket chart if at least 1 out of 5 fuzzing campaigns finds a bug. If we switched to requiring *all* fuzzing campaigns to find the bug, the results paint an even more compelling argument: *only* Heap-based pools consistently find counterexamples—we omit the graphs because all other options fail to consistently solve a task across runs, indicating a very high variance that heap seed pool does not exhibit. As a result, we plan to submit a PR to QuickChick to use the effective configuration as a default going forward.

6.4 Comparison of Integrated Shrinking with External Shrinking

Test case reduction, or counterexample minimization, or shrinking, is a fundamental aspect of random testing [Zeller and Hildebrandt 2002], however it garners much less attention than its random generation counterpart. A major splitting point in PBT frameworks has been their approach to shrinking, with the original QuickCheck using delta-debugging style structural shrinking where the generated counterexample structure is shrunk by removing its substructures to produce smaller versions with an accompanying reproducer that tests if the smaller versions keep triggering the bug; and Hypothesis [Maciver and Donaldson 2020] in Python, falsify [de Vries 2023] in Haskell, and RACKCHECK [Popa 2020] in Racket use an alternative route, instead of shrinking the generated structure, they shrink the source of randomness that led to the generation. This method, called integrated or internal shrinking, removes the requirement of a separate shrinking function and preserves the generation invariants in the generator while making no guarantees on the similarity of the shapes obtained in the shrinking phase.

We argue, as we have done many times in the paper, that such splitting points should not be decisions at the level of the library, but rather decisions to be made in relevant contexts by the users of the library. In an ideal scenario, a user should be able to switch between using an integrated shrinker versus an external one, and vice versa; and perhaps even use or build a new approach that might not be available when the library was written in the first place. Instead of forcing the users to choose or change libraries based on their context and approach to shrinking, DBAS allows for building your own property runner with its custom approach to shrinking, and that is precisely what we have done in this experiment.

We have compared the effectiveness of the integrated shrinker of RACKCHECK against a simple external shrinker we have implemented in our DBAS PBT library in Racket. We have conducted our experiment on the System F workload in ETNA, and we used the same generator in both RACKCHECK and DBAS-style Racket library, equipping our library with a simple type-based external shrinker we implemented in place of the integrated shrinker.

Our results show that the external shrinker successfully shrinks System F terms to a minimal counterexample that is an average of 2.66 times smaller than the original input with a standard deviation of 1.23, while the integrated shrinker of RACKCHECK only shrinks the inputs to an average rate of 1.04 times smaller than the original input with a standard deviation of 0.37. RACKCHECK only shrunk the inputs to smaller inputs in 66 out of the 360 trials, kept the size the same in 77, grew the inputs in 68, and failed to shrink in 149 trials. Fig. 20 depicts these results, which might be surprising at first glance: does the internal shrinker really only shrink the inputs in 20% of the trials? It turns out that in this case, it does. Its notion of size is based on the structure of the generator rather than on the input itself, so "smaller randomness" may not actually lead to smaller input values.

The point of this experiment is not to demonize internal shrinking—for many testing situations, it is perfectly sufficient and much more user-friendly than a bespoke shrinker. However, in pathological cases, programmers need an escape hatch, and DBAS provides the necessary configurability.

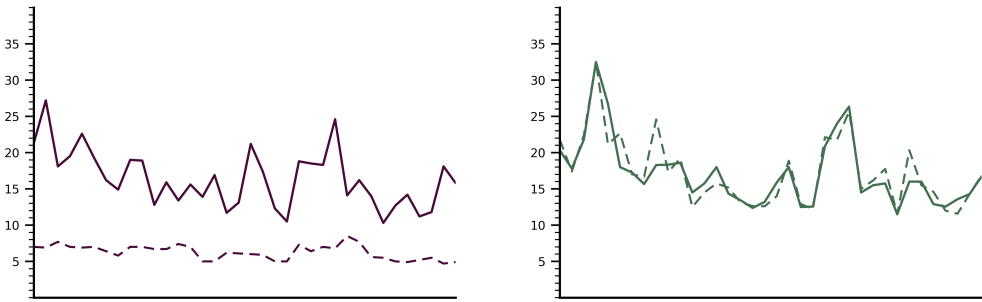


Fig. 20. Original sizes (continuous) and shrunk sizes (dashed) across tasks (x axis) for the external shrinker (left) and internal shrinker (right).

6.5 Parallelizing Property-Based Testing

An important recent work on novel property runners is QuickerCheck [Krook et al. 2024], where authors implement and evaluate a parallel run-time for QuickCheck [Claessen and Hughes 2000], achieving massive performance gains on a variety of workloads. The underlying idea the authors propose is rather simple, they provide an alternative, parallel property runner quickCheckPar, where multiple worker threads share a common variable that controls the size of the generated inputs. Using this strategy, the authors achieve an almost linear speed-up with respect to the number of physical cores used in testing. Unfortunately, due to the rigid structure of shallow embedding based PBT libraries, creating such a parallel runner requires intensive engineering effort. The commit history of the project shows that the implementation of this new runner is an effort spanning 2 years and more than 50 commits. In contrast, the DBAS-style Racket library let us implement a naive version of the QuickerCheck algorithm as a user-space runner, without changing the property representation or the core framework. Our implementation uses worker

threads with 2 shared atomic variables, one keeping the current number of tests across threads, and one indicating if the process of testing is finished or not.

We have compared this parallel runner with 4 threads against the single threaded runner across 131 tasks (53 BST, 58 RBT, 20 STLC). As the results of the simple generational property runner evaluation in Racket (§ 6.2.2) shows, the majority of the tasks in ETNA [Shi et al. 2023] are trivial for bespoke generators, they can be solved within 0.1 seconds. In order to measure the impact of the parallelization in the presence of such variation across tasks, we apply a set of cutoffs of time differences between task solving performance, and report the average ratio of time to solve for each cutoff.

- For all tasks where the difference between time-to-failure for single and parallel runners is greater than a millisecond, the average of their ratios is 1.2, where the parallel runner takes 20% more time *on average* than the single threaded runner across 49 of the 131 tasks.
- For all tasks where the difference between time-to-failure for single and parallel runners is greater than 0.1 seconds, the average of their ratios is 0.33, where the parallel runner is *on average* 3 times faster than the single threaded runner across 10 of the 131 tasks.
- Only 1 task has a difference of more than 1 second, which the average time-to-failure for single threaded runner is 3.56 seconds, and the average time-to-failure for the parallel runner is 1.16 seconds, resulting in again a 3 times speed-up due to parallelization.

7 Related Work

Throughout this paper, we have thoroughly discussed various property-based testing frameworks, their property languages, and the property runners that they come bundled with. Here, we briefly summarize related work in PBT and we also discuss related work in deeply embedded domain-specific languages.

Property-Based Testing Frameworks. To our knowledge, no widely-used frameworks expose a reified property representation that supports user-defined runners without modifying library internals. However, there are multiple frameworks that make distinctly different choices in what capabilities they provide to users. Table 1 summarizes the status quo in popular tools.

Table 1. Shrinking and feedback options in selected set of widely used frameworks; * denotes experimental or partial support.

Framework	Language	Shrinking	Feedback
QuickCheck [Claessen and Hughes 2000]	Haskell	External	
Hedgehog [Stanley 2019]	Haskell	Internal	
QuickCheck [Paraskevopoulou et al. 2015]	Rocq	External	Coverage [Lampropoulos et al. 2019]
Hypothesis [MacIver 2016]	Python	Internal [MacIver and Donaldson 2020]	Coverage*
Zest [Padhye et al. 2019b]	Java	AFL Trimming	Customizable [Padhye et al. 2019d]
QCheck	OCaml	Both*	
Crowbar	OCaml	AFL Trimming	Coverage
RackCheck	Racket	Internal	
QuviQ QuickCheck	Erlang	Both	
PropEr	Erlang	Both	Targeting [Löschner and Sagonas 2017]

Free Generators. The design of our deeply embedded property language builds on a rich literature of embedded DSLs [Hudak 1996]. In particular, our approach parallels work on *free generators* [Goldstein and Pierce 2022], which present a deeply embedded DSL for writing random generators. Deeply embedded properties and generators are largely orthogonal—free generators may be able to simplify the implementation of some of the different runners described in § 6, but they do not allow the

developer to re-program the loop itself. This follows a more general trend of using free structures to increase expressivity or usability in the programming languages community. For example, itrees [Xia et al. 2020] introduced a general-purpose Rocq structure which is essentially a coinductive variant of free monads, which allows them to represent and reason about interactive recursive programs. In follow up work, Li and Weirich [2022] explored how other free structures (such as applicative functors) can be used as an alternative to free-monad-based embeddings.

Mixed Embeddings. There is also a long line of related work in attempting to bridge the benefits of shallow embeddings (ease of use, as in the current property language) with those of deep ones (extensibility). For example, Carrete et al. [2009] showed how to use typeclasses in Haskell to allow for shallow embeddings that can be interpreted in different ways, hinting at a way of incorporating a deeper property language in a type system such as Haskell's. More recently, Matsuda et al. [2023] showed how to convert between embedding representations by *unembedding* alleviating some of the problems with Higher-Order Abstract Syntax representations. Finally, Prinz et al. [2022] introduced a hybrid embedding where typing derivations are represented as a deep embedding indexed by shallow terms in the host language, offering pattern matching capabilities.

8 Conclusion and Future Work

We have presented Programmable Property-Based Testing, a novel PBT paradigm that changes the underlying representation of the properties using a mixed embedding, deferred binding abstract syntax (DBAS), we introduce. Programmable PBT is a new approach to encoding properties for PBT that enables more flexible and programmable testing. The key advance made by DBAS is to reify properties as a free data structure; allowing them to be written in a clear and readable way, separate from the property runner that tests them. These more deeply embedded properties can then be inspected and interpreted by user-defined property runners. With the help of DBAS, developers in Rocq, Racket, and hopefully soon other programming languages, can tailor, experiment, and adapt their testing setup to their domain.

In the future, we intend to make DBAS convenient to program in more mainstream languages than Rocq and Racket. On the static typing side, we chose to implement our approach in Rocq as its dependent type system and powerful notation mechanism allowed us to conveniently express the heterogeneous contexts involved. The same approach should also be implementable in Haskell using standard GADT-based heterogeneous lists [Yorgey et al. 2012]; however, without access to notational conveniences as in Rocq, it remains future work to see how to regain a similar degree of ergonomics in that setting. On the dynamic typing side, the techniques described should be readily transferable to other languages. For example, given Python's powerful reflection capabilities and its convenient decorator features, a deferred-binding style property language should be implementable on top of, e.g., Hypothesis.

9 Data Availability Statement

An artifact [Keles et al. 2026] containing instructions for reproducing all our experimental results along with the source code for the Programmable PBT libraries in Rocq and Racket is publicly available.

Acknowledgments

We thank our colleagues at the Programming Languages Research group at the University of Maryland, and the anonymous reviewers for their helpful comments.

The work by Alperen Keles and Leonidas Lampropoulos was funded by NSF award #2145649 CAREER: Fuzzing Formal Specifications. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation. The work by Harrison Goldstein was partially funded by the Victor Basili Postdoctoral Fellowship from the University of Maryland.

A Appendix

A.1 Custom-Feedback Guided (Targeted) Property Runner

In recent years, PBT tools and mutation based fuzzers have begun to find common ground. On one hand, fuzzing tools have been trying to move towards more and more structured generation of inputs as well as incorporate and encode more complex properties than simply “the program doesn’t crash” [Padhye et al. 2019c; Park et al. 2021; Sun et al. 2023]. On the other hand, we have seen a rise in the popularity of property-based testing tools that are able to guide the generation of inputs using feedback [Lampropoulos et al. 2019; Löschner and Sagonas 2017; Padhye et al. 2019b]. Following this trend, we have used DBAS to implement mutation-based generation, developing two property runners leveraging mutation and feedback.

A mutation-based targeted property runner has two important differences from the simple generational property runner described in Fig. 12: the feedback and the targeting. It uses a genetic algorithm based on a user provided custom feedback function for guiding the search towards interesting inputs, and it employs a user-provided or type-derived mutator functions to mutate the input it currently focuses on. The main benefit of such a runner is that it means developers can potentially avoid writing complex generators for complex types and preconditions.

The runner is further parameterized by a seed pool (to keep track of interesting inputs) and a utility function (to accommodate different types of feedback). As shown in the pioneering work of Padhye et al. [2019d], customizable feedback under user control can lead to very effective testing, and DBAS allows even more relevant choices to be made without needing to modify the internals of a framework. A pictorial depiction of the targeted property runner is illustrated in Fig. 6.

In Fig. 21, we provide an annotated Rocq implementation of the custom-feedback guided (targeted) property runner in Rocq. Similar to the simple generational runner or coverage-guided (fuzzing) runner in § 6, this runner reuses the building blocks we have provided for writing novel property runners.

It is important to note that this specific runner is not a fixed part of our library, but merely a default behavior that is convenient and can be readily used. Alternative implementations may change the feedback behavior to accommodate feedback in the discard cases, or change the printing or shrinking behaviors. Even the seed pool and utility typeclasses are provided as sensible defaults and building blocks, rather than static design choices as is the case in shallow embedding based PBT frameworks.

A.2 Parallel Property Runner

In Fig. 22 is a slightly abridged version of our Racket parallel testing runner. Parallelism is done through Racket futures. We use a lock-free shared counter and a flag that is set if a counterexample is found, which is necessary because Racket futures are not able to be halted arbitrarily. Each worker thread grabs a test number from the counter, and loops until it either finds a counterexample

```

Definition targetLoop (fuel : nat) (cprop : Prop 0)
  (feedback_function : {cprop} → Z) {Pool : Type}
  {pool : SeedPool} (seeds : Pool)
  (utility : Utility) : G Result :=
let fix targetLoop' (fuel : nat)
  (passed : nat) (discards : nat)
  (seeds : Pool) : G Result :=
match fuel with
| 0 => ret (mkResult discards false passed [])
| S fuel' =>
  let directive := sample seeds in
  input ← match directive with
  | Generate => gen cprop (log2 (passed + discards))
  | Mutate source => mutate cprop source
end;;
res <- run cprop input;;
match res with
| Normal seed false => (* Fails *)
  let shrunk := shrinkLoop 10 cprop seed in
  let printed := print cprop 0 shrunk in
  ret (mkResult discards true (passed + 1) printed)
| Normal seed true => (* Passes *)
  let feedback := feedback_function seed in
  match useful seeds feedback with
  | true =>
    let seeds' := invest (seed, feedback) seeds in
    targetLoop' fuel' (passed + 1) discards seeds'
  | false =>
    let seeds' := match directive with
    | Generate => seeds
    | Mutate source => revise seeds
    end in
    targetLoop' fuel' (passed + 1) discards seeds'
  end
| Discard _ _ => (* Discard *)
  targetLoop' fuel' passed (discards + 1) seeds
end
end in
targetLoop' fuel 0 0 seeds pool utility.

```

Fig. 21. Custom-Feedback Guided (Targeted) Testing Loop in Rocq

The structure of the targeted property runner depicted in Fig. 6 is also reflected in the structure of the targetLoop in Fig. 21. This loop performs all the usual bookkeeping we discussed in the simple loop of Fig. 12, but adds mutation and feedback mechanisms guiding the search towards interesting inputs. The loop is parameterized by the **feedback function**, the **seed pool**, and the **utility function**, where the seed pool can be configured with different data structures such as a priority, FIFO, or FILO queue, and the utility function can be configured with different strategies such as a threshold, or a more complex stateful model. The loop uses the Seed Pool and Utility typeclasses to orchestrate the search.

Walking through the runner, we see that it diverges from the simple generational property runner in Fig. 12 in its input generation, where it might either **generate** from scratch, or **mutate** by **sampling** the seedpool. This input is then **run** through the property, and the failure and discarded cases are handled exactly the same as the simple generational property runner. If the test succeeds, depending on the feedback calculated by the user-provided feedback function, the seed might be **invested** in the seed pool, or the seed pool might be **revised** to reduce the energy of the seed. The loop then continues with the updated seed pool, and the passed and discarded counts.

or the counter exceeds the test number, with the main thread waiting for results from the workers and combining them when they finish.

Like all the other loops we introduced, this loop is not a fixed part of the library, and more efficient or sophisticated parallel runners can be implemented without changing the underlying property representation.

```

(define (parallel-run-loop tests prop [num-workers (processor-count)])
  ; atomic counter for the test number
  (define counter (box 0))
  ; flag set if a thread finds a counterexample
  (define found-counterexample? (box #f))
  ; function called by each thread
  (define (worker-thunk)
    ; each thread creates its own random number generator
    (define rng (make-pseudo-random-generator))
    (let worker-loop ([passed 0]
                      [discards 0])
      ; fetch and increment the test number counter
      (define n (box-faa! counter 1))
      (cond
        ; if the number of tests has exceeded the total, return the thread results
        [(>= n tests) (results #f passed discards #f)]
        ; if another thread has found a counterexample, return the thread results
        [(unbox found-counterexample?) (results #f passed discards #f)]
        [else
         ; run a single test
         (let-values ([{res env} (gen-and-run prop run-rackcheck-generator rng n)])
           (case res
             ; if a counterexample was found, set the found flag
             ; and return the thread results
             [(fail)
              (set-box! found-counterexample? #t)
              (results #t passed discards env)]
             ; on pass or discard, increment the relevant counter and recur
             [(pass) (worker-loop (add1 passed) discards)]
             [(discard) (worker-loop passed (add1 discards))]))]))
    ; spawn workers
  (define workers
    (for/list ([_ (in-range num-workers)])
      (future worker-thunk)))
  ; read results from workers
  (for/fold ([res (results #f 0 0 #f)])
    ([worker workers])
    ; get results from this worker
    (define worker-res (touch worker))
    ; combine with previous worker results
    (results (or (results-foundbug? res) (results-foundbug? worker-res))
              (+ (results-passed res) (results-passed worker-res))
              (+ (results-discards res) (results-discards worker-res))
              (or (results-counterexample res) (results-counterexample worker-res)))))

```

Fig. 22. Parallel Property-Based Testing Loop in Racket

References

- Jinsheng Ba and Manuel Rigger. 2023. Testing Database Engines via Query Plan Guidance. In *The 45th International Conference on Software Engineering (ICSE'23)*.
- Marcel Böhme, Valentin J. M. Manès, and Sang Kil Cha. 2020. Boosting fuzzer efficiency: an information theoretic perspective. In *ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8-13, 2020*, Prem Devanbu, Myra B. Cohen, and Thomas Zimmermann (Eds.). ACM, 678–689. <https://doi.org/10.1145/3368089.3409748>
- Marcel Böhme, Van-Thuan Pham, and Abhik Roychoudhury. 2019. Coverage-Based Greybox Fuzzing as Markov Chain. *IEEE Trans. Software Eng.* 45, 5 (2019), 489–506. <https://doi.org/10.1109/TSE.2017.2785841>
- Marcel Böhme, Van-Thuan Pham, Manh-Dung Nguyen, and Abhik Roychoudhury. 2017. Directed Greybox Fuzzing. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (Dallas, Texas, USA) (CCS '17)*. Association for Computing Machinery, New York, NY, USA, 2329–2344. <https://doi.org/10.1145/3133956.3134020>
- Richard J. Boulton, Andy Gordon, Michael J. C. Gordon, John Harrison, John Herbert, and John Van Tassel. 1992. Experience with Embedding Hardware Description Languages in HOL. In *Proceedings of the IFIP TC10/WG 10.2 International Conference on Theorem Provers in Circuit Design: Theory, Practice and Experience (proceedings of the ifip tc10/wg 10.2 international conference on theorem provers in circuit design: theory, practice and experience ed.)*. North-Holland Publishing Co., 129–156. <https://www.microsoft.com/en-us/research/publication/experience-embedding-hardware-description-languages-hol/>
- Cameron Bytheway. 2019. *Bolero: A fuzzing and property testing front-end framework for Rust*. <https://github.com/camshaft/bolero>
- Jacques Carrete, Oleg Kiselyov, and Chung-Chieh Shan. 2009. Finally tagless, partially evaluated: Tagless staged interpreters for simpler typed languages. *Journal of Functional Programming* 19, 5 (2009), 509–543. <https://doi.org/10.1017/S0956796809007205>
- Jinfu Chen, Shengran Wang, Saihua Cai, Chi Zhang, Haibo Chen, Jingyi Chen, and Jianming Zhang. 2023. A Novel Coverage-guided Greybox Fuzzing based on Power Schedule Optimization with Time Complexity. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (Rochester, MI, USA) (ASE '22)*. Association for Computing Machinery, New York, NY, USA, Article 172, 5 pages. <https://doi.org/10.1145/3551349.3559550>
- Adam Chlipala. 2008. Parametric Higher-order Abstract Syntax for Mechanized Semantics. In *Proceedings of the 13th ACM SIGPLAN International Conference on Functional Programming (Victoria, BC, Canada) (ICFP '08)*. ACM, New York, NY, USA, 143–156. <https://doi.org/10.1145/1411204.1411226>
- Koen Claessen and John Hughes. 2000. QuickCheck: a lightweight tool for random testing of Haskell programs. In *Proceedings of the 5th ACM SIGPLAN International Conference on Functional Programming*. ACM, 268–279. <https://doi.org/10.1145/351240.351266>
- Koen Claessen and Michał Pałka. 2013. Splittable pseudorandom number generators using cryptographic hashing. In *ACM SIGPLAN Symposium on Haskell*. ACM, 47–58. http://publications.lib.chalmers.se/records/fulltext/183348/local_183348.pdf
- Edsko de Vries. 2023. falsify: Internal Shrinking Reimagined for Haskell. In *Proceedings of the 16th ACM SIGPLAN International Haskell Symposium (Seattle, WA, USA) (Haskell 2023)*. Association for Computing Machinery, New York, NY, USA, 97–109. <https://doi.org/10.1145/3609026.3609733>
- Stephen Dolan. 2017. Property Fuzzing for OCaml. <https://github.com/stedolan/crowbar>.
- Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Marc Heuse. 2020. AFL++: combining incremental steps of fuzzing research. In *Proceedings of the 14th USENIX Conference on Offensive Technologies (WOOT'20)*. USENIX Association, USA, Article 10, 1 pages.
- Harrison Goldstein, Joseph W. Cutler, Daniel Dickstein, Benjamin C. Pierce, and Andrew Head. 2024. Property-Based Testing in Practice. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24, Vol. 187)*. Association for Computing Machinery, Lisbon, Portugal, 1–13. <https://doi.org/10.1145/3597503.3639581>
- Harrison Goldstein, John Hughes, Leonidas Lampropoulos, and Benjamin C. Pierce. 2021. Do Judge a Test by its Cover: Combining Combinatorial and Property-Based Testing. In *Proceedings of the European Symposium on Programming (ESOP) (Lecture Notes in Computer Science)*. https://doi.org/10.1007/978-3-030-72019-3_10
- Harrison Goldstein and Benjamin C. Pierce. 2022. Parsing Randomness. *Proceedings of the ACM on Programming Languages* 6, OOPSLA2 (Oct. 2022), 128:89–128:113. <https://doi.org/10.1145/3563291>
- Alex Groce, Chaoqiang Zhang, Eric Eide, Yang Chen, and John Regehr. 2012. Swarm Testing. In *Proceedings of the 2012 International Symposium on Software Testing and Analysis (Minneapolis, MN, USA) (ISSTA 2012)*. ACM, New York, NY, USA, 78–88. <https://doi.org/10.1145/2338965.2336763>
- Zac Hatfield-Dodds and contributors. 2025. *HypoFuzz: Adaptive fuzzing of Hypothesis tests*. <https://github.com/ZacHD/hypofuzz>

- Paul Hudak. 1996. Building domain-specific embedded languages. *Comput. Surveys* 28, 4es (Dec. 1996), 196. <https://doi.org/10.1145/242224.242477>
- Alperen Keles. 2024. RackCheck Issue #14: fix shrinking of generators that use size. <https://github.com/Bogdanp/rackcheck/issues/14>.
- Alperen Keles, Justine Frank, Ceren Mert, Harrison Goldstein, and Leonidas Lampropoulos. 2026. *Programmable Property-Based Testing — ICFP 2026 Artifact*. <https://doi.org/10.5281/zenodo.20434849>
- Robert Krook, Nicholas Smallbone, Bo Joel Svensson, and Koen Claessen. 2024. QuickerCheck: Implementing and Evaluating a Parallel Run-Time for QuickCheck. arXiv:2404.16062 [cs.PL] <https://arxiv.org/abs/2404.16062>
- Leonidas Lampropoulos. 2018. *Random Testing for Language Design*. Ph.D. Dissertation. University of Pennsylvania. <https://lemonidas.github.io/pdf/Leo-PhD-Thesis.pdf>
- Leonidas Lampropoulos, Michael Hicks, and Benjamin C. Pierce. 2019. Coverage Guided, Property Based Testing. In *Proceedings of the ACM Conference on Object-Oriented Programming Languages, Systems, and Applications (OOPSLA)*. <https://doi.org/10.1145/3360607>
- Leonidas Lampropoulos, Zoe Paraskevopoulou, and Benjamin C. Pierce. 2018. Generating Good Generators for Inductive Relations. In *Proceedings of the ACM Conference on Principles of Programming Languages (POPL)*. <https://doi.org/10.1145/3158133>
- Caroline Lemieux, Rohan Padhye, Koushik Sen, and Dawn Song. 2018. PerfFuzz: automatically generating pathological inputs. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis (Amsterdam, Netherlands) (ISSTA 2018)*. Association for Computing Machinery, New York, NY, USA, 254–265. <https://doi.org/10.1145/3213846.3213874>
- Caroline Lemieux and Koushik Sen. 2018. FairFuzz: a targeted mutation strategy for increasing greybox fuzz testing coverage. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (Montpellier, France) (ASE '18)*. Association for Computing Machinery, New York, NY, USA, 475–485. <https://doi.org/10.1145/3238147.3238176>
- Yao Li and Stephanie Weirich. 2022. Program adverbs and Tlön embeddings. *Proceedings of the ACM on Programming Languages* 6, ICFP (2022). <https://doi.org/10.1145/3547632>
- Andreas Löscher and Konstantinos Sagonas. 2017. Targeted Property-Based Testing. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, 46–56. <https://doi.org/10.1145/3092703.3092711>
- David Maciver and Alastair F. Donaldson. 2020. Test-Case Reduction via Test-Case Generation: Insights from the Hypothesis Reducer (Tool Insights Paper). In *34th European Conference on Object-Oriented Programming (LIPICs, Vol. 166)*. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 13:1–13:27. <https://doi.org/10.4230/LIPICs.ECOOP.2020.13>
- David R. MacIver. 2016. Hypothesis: Property-Based Testing for Python. <https://hypothesis.works/>.
- Kazutaka Matsuda, Samantha Frohlich, Meng Wang, and Nicolas Wu. 2023. Embedding by Unembedding. *Proc. ACM Program. Lang.* 7, ICFP, Article 189 (Aug. 2023), 47 pages. <https://doi.org/10.1145/3607830>
- Segev Elazar Mittelman, Alvin Resnick, Ivan Perez, Alwyn Goodloe, and Leonidas Lampropoulos. 2023. Don't Go Down the Rabbit Hole: Reprioritizing Enumeration for Property-Based Testing. In *Proceedings of the ACM SIGPLAN International Symposium on Haskell*.
- Rohan Padhye, Caroline Lemieux, and Koushik Sen. 2019a. JQF: coverage-guided property-based testing in Java. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (Beijing, China) (ISSTA 2019)*. Association for Computing Machinery, New York, NY, USA, 398–401. <https://doi.org/10.1145/3293882.3339002>
- Rohan Padhye, Caroline Lemieux, Koushik Sen, Mike Papadakis, and Yves Le Traon. 2019b. Semantic Fuzzing with Zest. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, 329–340. <https://doi.org/10.1145/3293882.3330576>
- Rohan Padhye, Caroline Lemieux, Koushik Sen, Mike Papadakis, and Yves Le Traon. 2019c. Validity fuzzing and parametric generators for effective random testing. In *Proceedings of the 41st International Conference on Software Engineering: Companion Proceedings, ICSE 2019, Montreal, QC, Canada, May 25–31, 2019*. Joanne M. Atlee, Tevfik Bultan, and Jon Whittle (Eds.). IEEE / ACM, 266–267. <https://doi.org/10.1109/ICSE-COMPANION.2019.00107>
- Rohan Padhye, Caroline Lemieux, Koushik Sen, Laurent Simon, and Hayawardh Vijayakumar. 2019d. FuzzFactory: domain-specific fuzzing with waypoints. *Replication Package for "FuzzFactory: Domain-Specific Fuzzing with Waypoints"* 3, OOPSLA (Oct. 2019), 174:1–174:29. <https://doi.org/10.1145/3360600>
- Zoe Paraskevopoulou, Catalin Hritcu, Maxime Denes, Leonidas Lampropoulos, and Benjamin C. Pierce. 2015. Foundational Property-Based Testing. In *6th International Conference on Interactive Theorem Proving (ITP)*. https://doi.org/10.1007/978-3-319-22102-1_22
- Jiwon Park, Dominik Winterer, Chengyu Zhang, and Zhendong Su. 2021. Generative type-aware mutation for testing SMT solvers. *Proc. ACM Program. Lang.* 5, OOPSLA (2021), 1–19. <https://doi.org/10.1145/3485529>
- Theofilos Petsios, Jason Zhao, Angelos D. Keromytis, and Suman Jana. 2017. SlowFuzz: Automated Domain-Independent Detection of Algorithmic Complexity Vulnerabilities. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (Dallas, Texas, USA) (CCS '17)*. Association for Computing Machinery, New York, NY, USA,

- 2155–2168. <https://doi.org/10.1145/3133956.3134073>
- Bogdan Popa. 2020. RackCheck: Property-Based Testing for Racket. <https://docs.racket-lang.org/rackcheck/index.html>.
- Jacob Prinz, Alex Kavvos, and Leonidas Lampropoulos. 2022. Deeper Shallow Embeddings. In *13th International Conference on Interactive Theorem Proving (ITP) (Lecture Notes in Computer Science)*. <https://doi.org/10.4230/LIPICs.ITP.2022.28>
- John Regehr, Yang Chen, Pascal Cuoq, Eric Eide, Chucky Ellison, and Xuejun Yang. 2012. Test-case reduction for C compiler bugs. In *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '12, Beijing, China - June 11 - 16, 2012*. 335–346. <https://doi.org/10.1145/2254064.2254104>
- Jessica Shi, Alperen Keles, Harrison Goldstein, Benjamin C. Pierce, and Leonidas Lampropoulos. 2023. Etna: An Evaluation Platform for Property-Based Testing (Experience Report). *Proc. ACM Program. Lang.* 7, ICFP, Article 218 (aug 2023), 17 pages. <https://doi.org/10.1145/3607860>
- SQLancer contributors. 2022. *SQLancer: Automated testing to find logic and performance bugs in database systems*. <https://github.com/sqlancer/sqlancer>
- Jacob Stanley. 2019. Hedgehog: Release with Confidence. <https://hackage.haskell.org/package/hedgehog/>.
- Jingling Sun, Ting Su, Jiayi Jiang, Jue Wang, Geguang Pu, and Zhendong Su. 2023. Property-Based Fuzzing for Finding Data Manipulation Errors in Android Apps. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023, San Francisco, CA, USA, December 3-9, 2023*, Satish Chandra, Kelly Blincoe, and Paolo Tonella (Eds.). ACM, 1088–1100. <https://doi.org/10.1145/3611643.3616286>
- Li-yao Xia, Yannick Zakowski, Paul He, Chung-Kil Hur, Gregory Malecha, Benjamin C. Pierce, and Steve Zdancewic. 2020. Interaction trees: representing recursive and impure programs in Coq. *Proc. ACM Program. Lang.* 4, POPL (2020), 51:1–51:32. <https://doi.org/10.1145/3371119>
- Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 283–294. <https://doi.org/10.1145/1993498.1993532>
- Brent A. Yorgey, Stephanie Weirich, Julien Cretin, Simon Peyton Jones, Dimitrios Vytiniotis, and José Pedro Magalhães. 2012. Giving Haskell a promotion. In *Proceedings of the 8th ACM SIGPLAN Workshop on Types in Language Design and Implementation (Philadelphia, Pennsylvania, USA) (TLDI '12)*. Association for Computing Machinery, New York, NY, USA, 53–66. <https://doi.org/10.1145/2103786.2103795>
- A. Zeller and R. Hildebrandt. 2002. Simplifying and isolating failure-inducing input. *IEEE Transactions on Software Engineering* 28, 2 (2002), 183–200. <http://pag-www.gtisc.gatech.edu/courses/common/zeller-tse02.pdf>

Received 2026-02-19; accepted 2026-05-13